

OWASP Top

10:2025

Da teoria à prática

Demonstrando 4 das vulnerabilidades mais comuns da web — ao vivo, num app real.

Apresentação técnica · DemoBank (app propositalmente vulnerável)

O que é a OWASP?

- **O**pen **W**orldwide **A**pplication **S**ecurity **P**roject
- Fundação sem fins lucrativos, comunidade aberta
- Materiais, ferramentas e padrões de segurança gratuitos

O que é o Top 10?

- Documento de **conscientização** mais conhecido da área
- Os **10 riscos mais críticos** em aplicações web
- Baseado em dados reais + pesquisa com especialistas
- Atualizado a cada ~3-4 anos

Novidades da edição 2025

-  **A03 — Software Supply Chain Failures**
(evolui de "componentes desatualizados")
-  **A10 — Mishandling of Exceptional Conditions** (tratamento de erros/exceções)
-  **SSRF** foi absorvido por **A01 — Broken Access Control**
-  **Security Misconfiguration** subiu para **A02**
(era A05)

OWASP Top 10:2025 — a lista

#	Categoria
A01	Broken Access Control  <i>demo</i>
A02	Security Misconfiguration
A03	Software Supply Chain Failures 
A04	Cryptographic Failures  <i>demo</i>
A05	Injection  <i>demo</i>
A06	Insecure Design
A07	Authentication Failures  <i>demo</i>
A08	Software or Data Integrity Failures
A09	Security Logging & Alerting Failures

#	Categoria
---	-----------

A10	Mishandling of Exceptional Conditions 
-----	---

O alvo: DemoBank

- App PHP + SQLite, propositalmente inseguro
- Já populado: 5 usuários, produtos, mensagens privadas
- Roda local: `php -S localhost:8000`

```
admin / admin      (administrador)
alice / 123456      bob / senha123
carla / qwerty      diego / gatinho
```

A05 — Injection SQL Injection

O problema

A entrada do usuário entra **direto** na query:

```
// search.php – VULNERÁVEL  
$q = $_GET['q'];  
$sql = "SELECT id, name, price  
      FROM products  
      WHERE name LIKE '%$q%'";  
db()->query($sql);
```

O banco não distingue **dado** de **comando**.

Demo ao vivo

1. Busca normal: mouse
2. Bypass do filtro: ' OR '1'='1
3. **Roubar os hashes:**

```
%' UNION SELECT id,username,password_md5 FROM users --
```

3 colunas na query original → 3 colunas no UNION. O - - comenta o resto.

Como corrigir

```
// Prepared statement: dado nunca vira comando
$stmt = db()->prepare(
    'SELECT id,name,price FROM products WHERE name LIKE ?'
);
$stmt->execute(["%$q%"]);
```

- **Sempre** prepared statements / queries parametrizadas
- Validação de entrada + menor privilégio no banco
- Nunca exibir erro de SQL para o usuário

A04 —

Cryptographic Failures

Hashes fracos

O problema

```
// Senha guardada assim:  
md5($password) // ex.: "123456" -> e10adc3949ba59abbe56e057f20f883e
```

- **MD5** é rápido demais → bilhões de tentativas/segundo
- **Sem salt** → hashes iguais para senhas iguais
- Já existem **bancos prontos** (rainbow tables) na internet

Demo ao vivo

1. Copiar o hash

e10adc3949ba59abbe56e057f20f883e

2. Colar em **crackstation.net** (ou hashes.com)

3. Resultado instantâneo: **123456**

```
Offline: john --format=raw-md5  
--wordlist=... hashes.txt
```

Como corrigir

```
// Hash lento, com salt automático  
$hash = password_hash($password, PASSWORD_ARGON2ID);  
password_verify($password, $hash); // no login
```

- Use **bcrypt** / **scrypt** / **Argon2** (lentos, com salt)
- TLS sempre; nada de dado sensível em texto plano
- Não invente criptografia própria

A07 —

Authenticati on Failures

Login sem proteção

O problema

-  Sem **rate limit** / atraso entre tentativas
-  Sem **bloqueio de conta** (lockout)
-  Sem **CAPTCHA** / MFA
-  Cadastro aceita **senha "1"**

→ brute force automatizado é trivial

Demo ao vivo

```
bash attack/bruteforce.sh admin http://localhost:8000
```

```
[-] password  
[-] qwerty  
[+] SENHA ENCONTRADA -> admin : admin
```

Como corrigir

- **Rate limiting** + bloqueio progressivo após N falhas
- **MFA** (segundo fator)
- Política de senha forte + checar contra senhas vazadas
- Mensagens de erro genéricas; logar tentativas (A09)

A01 —

Broken

Access Control

O risco nº 1

IDOR

O perfil vem do `id` da **URL**, sem checar dono:

```
// profile.php – VULNERÁVEL  
$id = (int) $_GET['id'];           // confia no cliente!  
$u  = "SELECT * FROM users WHERE id = $id";
```

`profile.php?id=2` → troco para `?id=1`, `?id=3...`
e vejo **CPF, saldo e mensagens privadas** de
qualquer um.

Falta de controle de função

```
// admin.php – VULNERÁVEL  
// (faltou checar o papel do usuário)  
$users = "SELECT * FROM users"; // qualquer logado acessa
```

Forced browsing: logado como alice, abro /admin.php e vejo o painel de admin completo.

Como corrigir

```
// Use a identidade da SESSÃO, não a da URL
$id = $_SESSION['uid'];

// Cheque papel em TODA rota sensível
if ($me['role'] !== 'admin') { http_response_code(403); exit; }
```

- Negar por padrão (deny by default)
- Verificação **server-side** de dono e de papel
- Nunca confiar em id/role vindos do cliente

A cadeia de ataque



1. **A05** SQLi na busca → extraio a tabela users com os hashes
2. **A04** Quebro o MD5 no crackstation → senhas em texto
3. **A07** (ou) brute force no login, sem rate limit
4. **A01** Logado, troco ?id= e abro /admin.php → dados de todos

Uma falha "pequena" raramente anda sozinha.

Boas práticas (resumo)

-  Prepared statements em **todo** acesso a dados
-  password_hash() (Argon2/bcrypt) + TLS
-  Rate limit, MFA, política de senha
-  Controle de acesso server-side, deny by default
-  Log e alerta (A09) + tratamento de erros (A10)
-  Revisar dependências / supply chain (A03)

Referências

- owasp.org/Top10/2025
- owasp.org/www-project-top-ten
- OWASP Cheat Sheet Series
- App da demo: [owasp-top10-demo/](#) (README com o passo a passo)

Obrigado! 

Perguntas?